STEP 0 → Workspace + rules

You are the lead software architect and senior full-stack engineer for a high-stakes disaster-intelligence hackathon project.
PROJECT NAME:
HIMALAYA-SHIELD
MISSION:
Build a professional AI-assisted disaster intelligence and emergency-response dashboard using the 26 August 2026 Nepal–China border disaster as the primary real-world demonstration case.
IMPORTANT ENGINEERING RULES:
1. Do not build a toy/demo-looking application.
2. Do not generate fake disaster facts.
3. Do not invent API responses, satellite measurements, population numbers, damage statistics, or scientific claims.
4. Any placeholder/demo value must be explicitly marked as DEMO/ILLUSTRATIVE.
5. Real-world factual information must be traceable to a source.
6. AI predictions must display confidence and data provenance where applicable.
7. Do not claim that AI can exactly predict earthquakes or glacier collapses.
8. Do not make autonomous life-or-death decisions. The system is decision-support software and must keep a human-in-the-loop.
9. Keep the architecture modular so future real-time satellite/weather/GIS APIs can replace prototype data without redesigning the frontend.
10. Do not over-engineer features that are not required for the hackathon MVP.
HACKATHON MVP:
Satellite evidence
→ change detection
→ affected-area visualization
→ damage/exposure assessment
→ risk scoring
→ emergency priority
→ response recommendation.
TECHNOLOGY TARGET:
Frontend:
- React
- Vite
- TypeScript
- Tailwind CSS
Mapping:
- MapLibre GL JS or Leaflet
- Prefer MapLibre if technically appropriate.
Backend:
- Python
- FastAPI
AI / computer vision:
- Python
- PyTorch
- OpenCV
- appropriate pretrained/change-detection/segmentation models where practical
Geospatial:
- GeoPandas
- Rasterio
- GDAL
Database:
- PostgreSQL/PostGIS architecture, but do not make database setup block the initial frontend MVP.
VERSION CONTROL:
- Git
- clean commits
- meaningful commit messages
FIRST TASK:
Do NOT start implementing the complete application yet.
Instead:
1. Inspect the current workspace.
2. Determine whether the directory is empty or contains an existing project.
3. Propose the exact project architecture.
4. Create a PROJECT_ARCHITECTURE.md document containing:
   - system overview
   - frontend architecture
   - backend architecture
   - AI architecture
   - geospatial architecture
   - data flow
   - folder structure
   - API strategy
   - testing strategy
   - deployment strategy
   - security considerations
   - future extensibility
5. Create a DEVELOPMENT_RULES.md file containing the engineering and factual-integrity rules above.
6. Create a clear TODO/implementation roadmap divided into:
   - MVP
   - post-hackathon
   - long-term
7. Do not install unnecessary dependencies yet.
8. Do not create fake data disguised as real data.
9. Do not create the UI yet.
After completing this task:
- show me the proposed folder structure
- summarize the architecture
- list dependencies you recommend installing next
- report any assumptions you made
- report any uncertainties
- do not proceed to the next implementation stage automatically.

STEP 1 → Architecture

Now implement ONLY the frontend foundation described in PROJECT_ARCHITECTURE.md.
OBJECTIVE:
Create a production-quality emergency-command-center interface for HIMALAYA-SHIELD.

DO NOT implement AI, backend APIs, real satellite ingestion, authentication, or database functionality yet.
DESIGN DIRECTION:
The interface must look like a serious emergency operations / geospatial intelligence platform, NOT a generic SaaS dashboard.
Visual characteristics:
- dark cinematic command-center aesthetic
- professional typography
- high information density without clutter
- large map-first layout
- subtle borders and glass/layered panels
- restrained animations
- clear severity hierarchy
- accessible contrast
- responsive desktop-first design
- polished empty/loading/error states
PRIMARY NAVIGATION:
1. Dashboard
2. Satellite Analysis
3. Damage Assessment
4. Risk Analysis
5. Emergency Response
DASHBOARD:
Create:
- top status bar
- system status indicator
- current event indicator
- large map container
- severity legend
- KPI cards
- priority-zone panel
- recent analysis/activity panel
MAP LAYERS UI:
- Satellite
- Flood
- Landslide
- Damage
- Buildings
- Roads
- Hospitals
- Population
Use clearly labelled placeholder states where real data does not yet exist.
CRITICAL:
Do NOT fabricate real disaster statistics.
If you need values for visual development, use values such as:
"—"
"Awaiting analysis"
"Demo value"
rather than pretending they are real.
COMPONENT REQUIREMENTS:
Create reusable components:
- AppShell
- Sidebar
- TopBar
- MapPanel
- KPIGrid
- RiskIndicator
- SeverityLegend
- PriorityList
- DataSourceBadge
- ConfidenceBadge
- LoadingState
- EmptyState
- ErrorState
ARCHITECTURE:
Keep components modular.
Do not put the entire application into App.tsx/App.jsx.
Use TypeScript interfaces/types for structured data.
ANIMATION:
Use subtle transitions only.
No excessive animations.
No distracting particle effects.
ACCESSIBILITY:
- keyboard navigation
- readable contrast
- semantic buttons
- tooltips where useful
RESPONSIVE:
Desktop is the primary target because this is a hackathon command dashboard, but the layout must degrade gracefully on smaller screens.
QUALITY BAR:
The first screenshot should look credible enough to be shown to a government emergency-management stakeholder.
After implementation:
1. Run the frontend.
2. Run lint/type checks.
3. Fix all errors.
4. Verify every navigation item.
5. Verify responsive behavior.
6. Report exactly what files changed.
7. Do not implement the next stage automatically.

STEP 2 → UI foundation

Now implement the HIMALAYA-SHIELD geospatial map foundation.
OBJECTIVE:
Turn the dashboard map placeholder into a real interactive map centered on Nepal and the Himalayan region.
TECHNOLOGY:
Use MapLibre GL JS if compatible with the current project.

If MapLibre creates unnecessary complexity, use Leaflet.
MAP REQUIREMENTS:
- Nepal-focused initial viewport
- smooth zoom/pan
- terrain-aware visual hierarchy where available
- professional dark basemap
- attribution
- zoom controls
- layer controls
CREATE THESE LAYERS:
1. Satellite
2. Flood
3. Landslide
4. Damage
5. Buildings
6. Roads
7. Hospitals
8. Population
For now, use safe prototype layers where real datasets are not yet connected.
IMPORTANT:
Do NOT fabricate disaster polygons and present them as actual 2026 observations.
If prototype geometry is required:
- clearly label it as DEMO
- keep it geographically plausible
- isolate it inside a dedicated demo-data module
- make it easy to replace with real GeoJSON later.
CREATE:
- map configuration module
- layer registry
- marker configuration
- popup component
- legend
- map loading state
- map error state
The map must be reusable from:
Dashboard
Satellite Analysis
Damage Assessment
Risk Analysis
Emergency Response
Do not duplicate map logic across pages.
QUALITY REQUIREMENT:
The map must visually dominate the dashboard.
After implementation:
- test zoom
- test pan
- test every layer toggle
- test popups
- test page navigation
- run lint/type checks
- fix errors
- report changes
- stop.

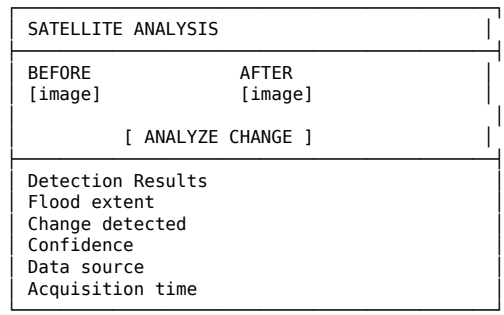
STEP 3 → Interactive map

Now add the REAL-WORLD CASE STUDY layer for the 26 August 2026 Nepal–China border disaster.
FACTUAL-INTEGRITY REQUIREMENT:
Use only information that can be verified from authoritative sources.
Preferred sources:
- USGS
- ESA
- Copernicus
- UNOSAT
- Nepal government/BIPAD/DHM
- other official sources where appropriate
DO NOT invent coordinates, affected-area boundaries, casualties, infrastructure damage, warning times, or scientific measurements.
Create a dedicated:
src/data/events/
module for the case study.
Create an event object containing:
- event name
- date
- country/region
- event type
- source references
- confidence/status
- description
- geographic extent ONLY where verified
Create a:
DataSourceBadge
component showing the source of important information.
Create:
EventSummaryPanel
It should display concise factual information.
IMPORTANT:
Where a scientific explanation remains preliminary, explicitly label it:
"Preliminary"
or
"Mechanism under investigation"
Do NOT phrase preliminary information as established fact.
Add the event to the map.
Add:
- event marker
- affected-region layer only when supported by actual source data

- event information popup
Create a SOURCE_REGISTRY.md documenting:
- source
- URL
- what information it supports
- date accessed
- whether the information is confirmed or preliminary
Do not scrape random websites.
After implementation:
- verify every displayed factual claim
- run lint/type checks
- inspect the UI
- report every external source used
- stop.

STEP 4 → Real Nepal disaster data

Now implement the HIMALAYA-SHIELD Satellite Analysis module.
OBJECTIVE:
Create a professional before/after satellite-analysis workflow using REAL disaster imagery where legally and technically available.
SUPPORTED DATA TYPES:
- Sentinel-1 SAR
- Sentinel-2 optical
- Landsat
- GeoTIFF
- PNG/JPEG preview imagery
INTERFACE:



FEATURES:
- side-by-side comparison
- synchronized zoom if possible
- opacity slider
- swipe comparison if practical
- metadata panel
- source attribution
- acquisition date
- confidence indicator
- detected-change overlay
DO NOT claim that optical imagery works through clouds.
DO NOT claim that satellite imagery proves structural safety.
Explain Sentinel-1 and Sentinel-2 differences in a compact UI information panel.
ANALYSIS ENGINE:
Create an abstraction:
SatelliteAnalysisService
with methods such as:
loadScene()
compareScenes()
detectChange()
calculateAffectedArea()
Initially the service may use a controlled prototype dataset, but its interface must be designed so a real processing backend can replace it.
IMPORTANT:
Do not hide prototype/mock processing behind language implying that a real AI model executed.
If the prototype uses precomputed results, label:
"Precomputed analysis"
If actual inference runs, display:
"AI inference"
After implementation:
- test imagery loading
- test comparison controls
- test errors
- test metadata
- run lint/type checks
- stop.

STEP 5 → Satellite analysis

Now implement the Damage Assessment module.
OBJECTIVE:
Convert detected visual/geospatial changes into a structured damage-assessment view.
CATEGORIES:
- No significant detected change
- Possible damage
- Moderate damage
- Severe damage
- Destroyed / inaccessible
IMPORTANT:
These categories describe IMAGE/GIS evidence.
They must NOT be presented as certified structural engineering assessments.
CREATE:

DamageSummary
DamageMapLayer
DamageLegend
DamageDetailsPanel
ConfidenceIndicator
EvidencePanel
FOR EACH DETECTED FEATURE SHOW:
- feature ID
- category
- confidence
- evidence/source
- timestamp
- verification status
EXAMPLE:
Feature 184
Severe detected change
Confidence: 91%
Source: Sentinel imagery
Verification: Ground verification recommended
Do not fabricate statistics.
Create a service abstraction:
DamageAssessmentService
that can later accept:
- satellite imagery
- drone imagery
- model predictions
- GIS layers
Integrate the results into the map.
When a user clicks a damaged feature:
- highlight it
- open details
- zoom to it
- show evidence
After implementation:
- test all categories
- test confidence display
- test map interaction
- test empty/error states
- run checks
- stop.

STEP 6 → Damage + risk engine

Now implement the Risk Analysis module.
OBJECTIVE:
Create an explainable geospatial risk-assessment engine.
INPUT FACTORS:
- slope
- elevation
- rainfall
- geology
- satellite change
- historical hazard information
IMPORTANT:
Do not pretend the chosen weights are scientifically universal.
Make weights configurable.
Example prototype configuration:
exposureWeight
damageWeight
accessibilityWeight
secondaryHazardWeight
historicalRiskWeight
The UI must show WHY a zone received its score.
Example:
ZONE A
Overall Risk: HIGH
Factors:
Slope   HIGH
Rainfall   HIGH
Satellite change HIGH
Historical risk   MEDIUM
Accessibility   LOW
The final score must be presented as:
"Prototype risk score"
unless it is based on a validated scientific model.
CREATE:
RiskAnalysisService
RiskFactorCard
RiskBreakdown
RiskMapLayer
RiskScore
Use a transparent scoring function.
Do not create a black-box number with no explanation.
Integrate the risk result into the map.
After implementation:
- verify calculations
- test edge cases
- test missing data
- ensure missing data does not silently become zero
- run lint/type checks
- stop.

STEP 7 → Emergency response

Now implement the Emergency Response module.
OBJECTIVE:
Transform hazard + damage + exposure + accessibility information into an explainable RESPONSE PRIORITY.
The system must NOT autonomously order real emergency operations.
It provides decision support to human responders.
PRIORITY FACTORS:
- population exposure
- damage severity
- critical infrastructure
- accessibility
- secondary hazard risk
- medical urgency where available
Create:
ResponsePriorityService
Output:
Zone
Priority Score
Severity
Population Exposure
Infrastructure Impact
Accessibility
Recommended Action
Confidence
Verification Status
Example UI:
 PRIORITY 1
Zone A
Score: 94/100
Why?
- high population exposure
- severe detected damage
- limited road accessibility
- critical infrastructure nearby
Recommendation:
"Immediate field verification and rescue-resource assessment."
IMPORTANT:
Use language such as:
"Recommended"
"Prioritize for assessment"
"Human verification required"
Never:
"AI orders evacuation."
Add map visualization:
- priority 1
- priority 2
- priority 3
Add route visualization only if reliable road data is available.
If routing data is unavailable, explicitly state:
"Route analysis unavailable."
After implementation:
- test ranking
- test missing inputs
- verify explainability
- stop.

STEP 8 → Backend integration

Now implement the FastAPI backend required by the current frontend.
OBJECTIVE:
Create a clean backend API that exposes the disaster-intelligence services without tightly coupling the frontend to implementation details.
ARCHITECTURE:
backend/
  app/
    main.py
    api/
    models/
    schemas/
    services/
    core/
    data/
    tests/
CREATE API GROUPS:
/api/events
/api/satellite
/api/damage
/api/risk
/api/exposure
/api/response
Use Pydantic schemas.
Requirements:
- typed request/response models
- validation
- structured error responses
- logging
- CORS configuration
- environment variables
- no secrets in source code
IMPORTANT:
Do not create fake API endpoints that pretend to perform AI inference.
If an endpoint currently returns precomputed prototype data, explicitly identify it as prototype data in the schema/metadata.
Create:
GET /health
Create backend unit tests.
Frontend must consume the API through a dedicated client/service layer.

Do not put fetch() calls throughout React components.
After implementation:
- start backend
- test /health
- test every API endpoint
- run tests
- connect frontend
- fix errors
- stop.

STEP 9 → Testing + hardening

Perform a COMPLETE PRE-HACKATHON ENGINEERING AUDIT of HIMALAYA-SHIELD.
You are now acting as:
- senior software engineer
- QA engineer
- security reviewer
- UX reviewer
- geospatial application reviewer
- hackathon demo reviewer
DO NOT add random new features.
AUDIT:
1. Frontend
2. Backend
3. API contracts
4. Map rendering
5. Satellite workflow
6. Damage workflow
7. Risk calculations
8. Exposure calculations
9. Response prioritization
10. Loading states
11. Empty states
12. Error handling
13. Mobile/responsive behavior
14. Accessibility
15. Performance
16. Security
17. Source attribution
18. Factual claims
19. Prototype/demo labels
20. Console errors
21. Network errors
22. TypeScript errors
23. Python errors
24. Test coverage
25. Build process
FACTUAL-INTEGRITY AUDIT:
Search the project for:
- fabricated statistics
- unsupported casualty numbers
- fake satellite claims
- fake AI confidence
- claims of exact prediction
- claims of autonomous decision-making
- undocumented external data
- prototype data presented as real
Replace or clearly label anything unsafe or unsupported.
PERFORMANCE:
- optimize unnecessary renders
- lazy-load heavy modules where appropriate
- avoid huge assets
- avoid unnecessary dependencies
UX:
The judge should understand the system within 15 seconds.
The primary demo path must be:
Dashboard
→ Nepal event
→ satellite evidence
→ analysis
→ damage
→ risk
→ exposure
→ response priority
Finally:
1. run production build
2. run frontend tests
3. run backend tests
4. run lint/type checks
5. fix all errors
6. provide a final audit report
7. list remaining limitations
8. do NOT claim anything is complete unless it actually works.

STEP 10 → Final hackathon polish

Now optimize HIMALAYA-SHIELD specifically for a 3-minute live hackathon demonstration.
Do not change the underlying architecture.
Create a deterministic demo path that can be executed reliably without depending on an unpredictable external API.
The demo must show:
1. Real Nepal 2026 event
2. Satellite before/after evidence
3. Change detection
4. Damage visualization

5. Risk factors
6. Exposed assets
7. Priority ranking
8. Emergency recommendation
9. Source and confidence
Add a clearly visible:
"DEMO / LIVE DATA / PRECOMPUTED ANALYSIS"
status wherever appropriate.
Ensure that a temporary internet/API failure cannot destroy the entire presentation.
Use graceful fallback states.
Make the UI polished enough for a professional disaster-management presentation.
Do not fabricate results to make the demo look better.
Run the complete production build after the changes.

